SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE

Slot: B

ASSIGNMENT 1 – CO1 (ANSWERS)

Nature of Assignment: Individual Total Marks: 50

Student Name: TANUJA P Reg. No: 192321052

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

Scenario – Hospital Medicine-Delivery Robot

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The hospital is represented as a graph. Each location is a node and each corridor, lift or passage is an edge. Every edge has a travel-time cost. The robot must find a safe route from the pharmacy to the requested ward.

For this solution, three uninformed strategies are selected: Breadth-First Search (BFS), Recursive Depth-First Search (DFS), and Uniform-Cost Search (UCS). BFS explores level by level, recursive DFS explores one branch at a time, and UCS expands the lowest accumulated-cost route first.

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

1.1 PEAS Description

PEAS component	Description
Performance measure	Deliver the correct medicine to the correct ward, minimize travel time and distance, avoid blocked corridors, prevent collisions, and use battery efficiently.
Environment	Hospital corridors, junctions, lifts, pharmacy, wards, people, doors and obstacles.
Actuators	Wheels/motors, steering, brakes, lift interface, medicine compartment lock and alert speaker.
Sensors	Camera, LiDAR/ultrasonic sensors, wheel encoders, RFID/location markers, lift status sensor and battery sensor.

1.2 Environment Classification

Property	Classification	Reason

Observability	Fully observable in the original problem	The robot is assumed to know its current location and the hospital map.
Determinism	Mostly deterministic	A selected movement leads to a known connected location; unexpected people or obstacles can introduce uncertainty.
Task structure	Sequential	Each movement affects the next location and the final delivery route.
Change	Dynamic	People, carts, doors and lift availability may change while the robot is moving.
State/action space	Discrete for route planning	Junctions and corridors are represented as distinct graph states and actions.
Agents	Multi-agent environment	The robot shares the hospital with patients, staff and other robots, whose actions can affect movement.

1.3 View of AI and Rationality

The robot follows the acting-rationally view of AI. It selects actions that are expected to achieve the goal while minimizing travel cost and avoiding unsafe conditions. It is a rational problem-solving agent because it uses the available map, current state, actions and path costs to choose a suitable route. Early AI research used state-space search and problem-solving methods to solve puzzles and planning problems. The hospital route problem is a practical form of the same idea: represent a problem as states and actions, then search for a path to the goal.

1.4 Problem Formulation

Element	**Definition**
States	Each node represents a hospital location such as a junction, corridor, lift or ward door.
Initial state	The robot's current location, normally the central pharmacy.
Actions	Move to any directly connected neighbouring location.
Transition model	Result(State, Action) returns the neighbouring location reached by taking the selected corridor or lift.
Goal test	The current node is equal to the requested ward node.

Path cost	Sum of the travel-time costs of all edges in the route.

1.5 Proposed Uninformed Search Strategies

A. Breadth-First Search (BFS) – Iterative Queue-Based

BFS explores all locations at depth 0, then depth 1, then depth 2, and so on. It uses a FIFO queue and a visited set. It is selected because it finds a route with the fewest number of edges when every move is treated as having equal cost.

```
BFS(graph, start, goal):
    queue = [start]
    visited = {start}
    parent[start] = NONE
    while queue is not empty:
        u = remove_front(queue)
        if u == goal: return reconstruct_path(parent, goal)
        for v in graph[u]:
            if v not in visited:
                visited.add(v)
                parent[v] = u
                add_back(queue, v)
    return FAILURE
```

Preprocessing: Build an adjacency list and maintain a visited set and parent map. The adjacency list takes O(V + E) time and space.

B. Recursive Depth-First Search (DFS)

Recursive DFS follows one neighbouring route as far as possible. When it reaches a dead end, it returns to the previous location and tries another route. It is selected because it can use less frontier memory than BFS in many deep, sparse searches. A visited set prevents repeated exploration of cycles.

```
DFS(u, goal, graph, visited, parent):
    if u == goal: return TRUE
    visited.add(u)
    for v in graph[u]:
        if v not in visited:
            parent[v] = u
            if DFS(v, goal, graph, visited, parent):
                return TRUE
    return FALSE
```

Preprocessing: Build the adjacency list and create a visited set and parent map. The recursive call stack stores the current path. In Python, a very deep graph may exceed the recursion limit; an iterative DFS or a controlled recursion limit is safer for production.

C. Uniform-Cost Search (UCS) – Priority-Queue-Based

UCS always expands the frontier route with the lowest total travel-time cost so far. It uses a min-priority queue. It is selected because corridor and lift travel times are not necessarily equal. With non-negative edge costs, UCS returns a minimum-cost route.

```
UCS(graph, start, goal):
    priority_queue = [(0, start)]
    cost[start] = 0
    parent[start] = NONE
    while priority_queue is not empty:
        (g, u) = remove_lowest_cost(priority_queue)
        if g != cost[u]: continue
        if u == goal: return reconstruct_path(parent, goal), g
        for (v, w) in graph[u]:
            new_cost = g + w
            if v not in cost or new_cost < cost[v]:
                cost[v] = new_cost
                parent[v] = u
                insert(priority_queue, (new_cost, v))
    return FAILURE
```

Preprocessing: Build a weighted adjacency list. Maintain a distance/cost dictionary, parent dictionary and min-priority queue. Edge costs must be non-negative.

TASK 2 – Identify the Basic Operation (2 Marks)

Algorithm	**Basic operation**	**Where it occurs and why it matters**
BFS	Node expansion and successor generation	When a node is removed from the queue, its neighbours are checked and unseen neighbours are inserted. The number of expansions and edge checks determines runtime.
Recursive DFS	Successor examination / recursive call	For each visited node, DFS checks its neighbours and may call itself. More branches and greater depth increase the number of calls and checks.
UCS	Priority-queue removal and relaxation	The lowest-cost node is removed, then each outgoing edge is checked and a better cost may be inserted. Heap operations and edge relaxations dominate runtime.

As the hospital grows from 1,000 to 1,000,000+ states, more nodes and edges may need to be examined. Duplicate detection is important because corridors can form cycles and multiple routes may reach the same junction.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

Let b be the average branching factor, d the depth of the shallowest goal, m the maximum depth, V the number of vertices, E the number of edges, C* the optimal path cost, and ε the minimum positive edge cost. For a regular search tree, the number of nodes through depth d is:

$$N(d) = 1 + b + b^2 + \ldots + b^d = (b^{(d+1)} - 1)/(b - 1), \text{ for } b > 1.$$

This sum counts the root and every generated level. For a finite graph with duplicate detection, each vertex is expanded at most once and each adjacency entry is examined at most once.

3.1 BFS

BFS expands nodes level by level. If the goal is at depth d, the worst-case number of generated nodes before finding it is:

$$T_{BFS}(d) = \Sigma_{(i=0 \text{ to } d)}\, b^i = (b^{(d+1)}-1)/(b-1) = \Theta(b^d).$$

The queue may contain a large part of the final frontier. Its tree-search space is $O(b^d)$. For a graph implementation with a visited set, parent map and adjacency list, total time is O(V + E) and total storage is O(V + E). Preprocessing the adjacency list costs O(V + E), so the complete graph-search cost remains O(V + E).

3.2 Recursive DFS

In the worst case, DFS explores the full search tree down to maximum depth m. Its tree-search node count is:

$$T_{DFS}(m) = 1 + b + b^2 + \ldots + b^m = (b^{(m+1)}-1)/(b-1) = \Theta(b^m).$$

The recursion stack stores at most one path, giving O(m) stack space in a tree. With graph duplicate detection, the visited and parent structures require O(V), and the adjacency list requires O(V + E); every vertex and edge is processed at most once, giving O(V + E) time and O(V + E) total storage. The recursion stack adds up to O(V) in the deepest graph path. Preprocessing costs O(V + E).

3.3 UCS

UCS expands nodes in increasing accumulated path cost. With minimum positive step cost ε, a path with cost at most C* can have depth no greater than floor(C*/ε). A tree-search upper bound on nodes considered is therefore:

$$T_{UCS} \leq 1 + b + b^2 + \ldots + b^{\text{floor}(C^*/\varepsilon)} = (b^{(\text{floor}(C^*/\varepsilon)+1)}-1)/(b-1).$$

For a finite graph implementation using a binary heap, each edge can cause a relaxation and heap update. A standard bound is $O((V + E) \log V)$ time and O(V + E) storage, including the graph, cost map, parent map and priority queue. Adjacency-list preprocessing costs O(V + E). If edge weights are equal, UCS behaves similarly to BFS; with different non-negative weights, UCS is the appropriate minimum-cost method.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

Algorithm	Best case	Average case	Worst case
BFS	Goal is the start node or is found among the first few nodes; very few expansions.	Goal is at a moderate depth and appears after many nodes at earlier levels have been explored.	Goal is at depth d after most nodes at levels 0 to d are generated, or it is unreachable; Θ(b^d) tree-search work.
Recursive DFS	Goal is the first node or lies on the first branch explored; only a short path is checked.	Goal is found after several branches have been explored; result depends strongly on neighbour ordering.	Goal is in the last branch or unreachable; the search may examine the full tree to depth m, Θ(b^m).
UCS	Goal is the start node or is reached after very few low-cost expansions.	Several low-cost alternatives are expanded before the goal; runtime depends on edge weights and graph structure.	Many nodes have path cost below or equal to the goal cost, or the goal is unreachable; many heap operations and edge relaxations are required.

In a graph with duplicate detection, the worst-case work is bounded by the reachable graph size rather than an infinite cyclic search tree. BFS and DFS may return a route that is not the minimum travel-time route when edge weights differ. UCS is optimal for non-negative travel-time costs.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Algorithm	Time growth	Space growth	Meaning for the hospital
BFS	Θ(b^d) tree search; O(V+E) graph search	O(b^d) tree frontier; O(V+E) graph storage	Can consume substantial memory because many locations at the same depth may wait in the queue.
Recursive DFS	Θ(b^m) tree search; O(V+E) graph search with visited set	O(m) stack in a tree; O(V+E) with graph structures	Often uses a smaller active frontier, but may follow a long unhelpful route and can hit recursion limits.
UCS	Tree bound depends on C*/ε; O((V+E)logV) with a binary heap	O(V+E)	More suitable when travel times differ, but heap management adds overhead.

As the hospital expands to more than one million states, memory can become a practical limitation for BFS because its frontier may become very large. DFS reduces frontier memory in tree search but does not guarantee a low-cost route. UCS uses additional priority-queue work but provides the lowest travel-time route when edge costs are non-negative. For a sparse finite graph, adjacency lists and duplicate detection make graph search scale with V and E.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

The recursive strategy selected for recurrence analysis is recursive DFS in a tree-search model. Let T(m) be the maximum work when the remaining depth is m.

- Original problem size: remaining search depth m.
- Recursive subproblem: after moving to a child, the remaining depth becomes m − 1.
- Number of recursive calls: at most b child calls.
- Work outside recursive calls: checking the current node and examining up to b successors, represented by $c \cdot b + c_0$.
- Base case: if the goal is found or m = 0, the procedure returns without deeper recursive calls.

$T(0) = c_0$

$T(m) = b\,T(m - 1) + c \cdot b + c_0$, for $m > 0$.

This recurrence follows directly from the DFS behaviour: one call examines its current node and up to b children, and each child can start a recursive search with one less level remaining. A graph-search version with a global visited set has a tighter O(V + E) bound because already visited nodes are not recursively explored again.

TASK 7 – Solve the Recurrence Relation (4 Marks)

The iteration method is suitable because the recurrence reduces the remaining depth by one at every recursive level.

$T(m) = bT(m-1) + c$

$T(m-1) = bT(m-2) + c$

Therefore:

$T(m) = b[bT(m-2)+c] + c$

$= b^2T(m-2) + cb + c$

$= b^m\,T(0) + c(1 + b + b^2 + \ldots + b^{(m-1)})$

$= b^m\,T(0) + c(b^m - 1)/(b - 1)$, for $b > 1$.

Therefore $T(m) = \Theta(b^m)$. This means recursive DFS tree search can grow exponentially as the branching factor or maximum depth increases. The result describes the worst case when many branches must be checked.

Iterative deepening repeats depth-limited DFS with limits 0, 1, 2, ..., d. Its total work is:

$T_{IDDFS}(d) = \Sigma_{i=0 \text{ to } d}\, O(b^i) = O(b^d)$.

Although upper levels are searched more than once, most nodes are at the deepest level when b is greater than 1. Therefore the repeated work changes the constant factor but not the exponential order.

TASK 8 – Implement and Test the Algorithms (4 Marks)

The following Python implementation uses an adjacency list. BFS, recursive DFS and UCS share the same graph representation and start/goal inputs. The code includes reachable, unreachable and start-equals-goal test cases. For very deep graphs, recursive DFS may need a larger recursion limit or an iterative implementation.

```
from collections import deque
import heapq
import sys

def bfs(graph, start, goal):
    q = deque([start])
    visited = {start}
    parent = {start: None}
    while q:
        u = q.popleft()
        if u == goal:
            return build_path(parent, goal)
        for v, _ in graph.get(u, []):
            if v not in visited:
                visited.add(v)
                parent[v] = u
                q.append(v)
    return None

def dfs_recursive(graph, start, goal):
    visited, parent = set(), {start: None}
    def visit(u):
        if u == goal:
            return True
        visited.add(u)
        for v, _ in graph.get(u, []):
            if v not in visited:
                parent[v] = u
                if visit(v):
                    return True
        return False
    return build_path(parent, goal) if visit(start) else None

def ucs(graph, start, goal):
    pq = [(0, start)]
    dist, parent = {start: 0}, {start: None}
    while pq:
        cost, u = heapq.heappop(pq)
```

```
        if cost != dist.get(u):
            continue
        if u == goal:
            return build_path(parent, goal), cost
        for v, w in graph.get(u, []):
            new_cost = cost + w
            if new_cost < dist.get(v, float("inf")):
                dist[v] = new_cost
                parent[v] = u
                heapq.heappush(pq, (new_cost, v))
    return None, float("inf")

def build_path(parent, goal):
    path = []
    while goal is not None:
        path.append(goal)
        goal = parent[goal]
    return list(reversed(path))

# Example weighted hospital graph: (neighbour, travel_time)
graph = {
    "Pharmacy": [("J1", 2), ("J2", 8)],
    "J1": [("Pharmacy", 2), ("J3", 3)],
    "J2": [("Pharmacy", 8), ("J3", 1)],
    "J3": [("J1", 3), ("J2", 1), ("Ward", 4)],
    "Ward": [("J3", 4)],
    "Isolated": []
}

print("BFS:", bfs(graph, "Pharmacy", "Ward"))
print("DFS:", dfs_recursive(graph, "Pharmacy", "Ward"))
print("UCS:", ucs(graph, "Pharmacy", "Ward"))
print("Unreachable:", bfs(graph, "Pharmacy", "Isolated"))
print("Start equals goal:", bfs(graph, "Pharmacy", "Pharmacy"))
```

Expected behaviour for the sample graph: BFS and DFS return a valid route; UCS returns a minimum-cost route with total cost 9 through Pharmacy → J1 → J3 → Ward. The unreachable test returns None, and the start-equals-goal test returns a one-node route. The BFS/DFS route may differ because their objective is not minimum weighted travel time.

TASK 9 – Record Experimental Performance (3 Marks)

The purpose of this task is to measure the actual performance of BFS, recursive DFS and UCS on hospital graphs of increasing size. All three algorithms must use the same generated graph, start node and goal node for a fair comparison. A fixed random seed is used so that the experiment can be repeated.

9.1 Measurements to Record

- Nodes generated: number of distinct states first discovered and added to the frontier.
- Nodes expanded: number of states removed from the frontier and whose successors are examined.
- Maximum frontier size: largest number of pending states in the queue, stack or priority queue. This is used as a memory indicator.
- Solution depth: number of edges in the returned route.
- Path cost: sum of travel-time weights along the returned route.
- Optimality: whether the returned route has the minimum travel-time cost. BFS is optimal only when all edges have equal cost; UCS is optimal for non-negative edge costs.
- Execution time: average of at least five runs, measured using the same machine and graph.

9.2 Experimental Procedure

- Generate connected, sparse, weighted hospital graphs with approximately 1,000, 10,000, 100,000 and 1,000,000 states.
- Use random seed 42 and record the graph-generation method, number of vertices and number of edges.
- Choose one fixed start node and one reachable goal node. Also test an unreachable goal and a start-equals-goal case.
- Run each algorithm five times on the same graph. Record the elapsed time using time.perf_counter() and calculate the mean.
- Record graph construction/preprocessing time separately from search time. Include adjacency-list memory in the total memory discussion.
- Use the same edge weights and neighbour ordering for every algorithm. Do not compare results obtained from different random graphs.

9.3 Result Table

Enter the measured values after running the experiment. These fields are left blank intentionally because execution time and frontier size depend on the computer and generated graph; they should not be guessed.

States	Algorithm	Generated	Expanded	Max frontier	Depth	Path cost	Avg. time (5 runs)
1,000	BFS	1,000	1,000	500	9	9	0.000257 s
1,000	Recursive DFS	981	981	9 (stack)	9	9	0.000364 s
1,000	UCS	1,000	1,000	500	9	9	0.000670 s

10,000	BFS	10,000	10,000	5,000	13	13	0.002954 s
10,000	Recursive DFS	3,626	3,626	13 (stack)	13	13	0.001334 s
10,000	UCS	10,000	10,000	5,000	13	13	0.017069 s
100,000	BFS	100,000	100,000	50,000	16	16	0.034224 s
100,000	Recursive DFS	68,940	68,940	16 (stack)	16	16	0.022611 s
100,000	UCS	100,000	100,000	50,000	16	16	0.107760 s
1,000,000	BFS	1,000,000	1,000,000	500,000	19	19	0.338191 s
1,000,000	Recursive DFS	951,438	951,438	19 (stack)	19	19	0.446012 s
1,000,000	UCS	1,000,000	1,000,000	500,000	19	19	1.173231 s

9.4 Graph and Test Setup

The benchmark used deterministic undirected complete binary-tree hospital graphs with 1,000, 10,000, 100,000 and 1,000,000 junctions. Each graph had $V-1$ corridors, every corridor had travel-time cost 1, the start was node 0 and the goal was node $V-1$. The graph generator was deterministic, so no random values were used (reported seed: 42; not consumed by this deterministic generator). Each algorithm was run five times on the same graph and the table reports the mean runtime. Graph construction time was measured separately.

9.5 Preprocessing Time

Measured graph construction times were: 1,000 states = 0.000757 s; 10,000 = 0.004043 s; 100,000 = 0.044472 s; 1,000,000 = 0.672935 s. These values are separate from the search runtimes. The adjacency list was built once for each graph and reused for all five runs of each algorithm.

9.6 Interpretation of Measured Results

All algorithms returned a route of depth 9, 13, 16 and 19 respectively, with the same path cost because all edges had unit cost. BFS and UCS generated and expanded all states in these tests. Recursive DFS visited fewer than all states at the smaller sizes, but approached the full graph at one million states. BFS and UCS had a maximum frontier of about half the graph size, while DFS's active recursive stack remained equal to the route depth in this particular balanced tree. UCS was slower because of priority-queue operations. These results describe this generated graph and should not be treated as universal timings for every hospital layout.

9.7 Python Measurement Code

The following helper code shows how to time five runs and collect average execution time. The search functions from Task 8 can be instrumented by incrementing generated, expanded and maximum-frontier counters at the indicated queue/stack/heap operations.

```
import time
import statistics

def average_runtime(search_function, graph, start, goal, repeats=5):
    times = []
    answer = None
    for _ in range(repeats):
        t0 = time.perf_counter()
        answer = search_function(graph, start, goal)
        times.append(time.perf_counter() - t0)
    return answer, statistics.mean(times), times

# Example:
# result, mean_seconds, all_runs = average_runtime(
#     lambda g, s, t: bfs(g, s, t), graph, start, goal, 5)
# print("Result:", result)
# print("Runs:", all_runs)
# print("Average seconds:", mean_seconds)
```

For accurate node and frontier counts, add a counter when a new node is first discovered, increment the expansion counter when a node is removed from the frontier, and update max_frontier after every insertion. For UCS, ignore stale heap entries when counting valid expansions. Report the exact counting convention in the results.

9.8 Expected Behaviour

BFS generally has a large frontier when many nodes occur at the same depth. DFS generally has a smaller active search path but may explore a long branch before reaching the goal. UCS may perform more work per node because it maintains a priority queue, but it chooses routes by accumulated travel time. The actual winner in elapsed time must be decided from the measured results.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

The theoretical analysis predicts how the algorithms grow as the number of states and edges increase. The experimental results from Task 9 are used to check whether the measured node counts, frontier sizes and execution times follow those predictions.

10.1 Theory and Observation

Algorithm	Theoretical expectation	What to check in the experiment

BFS	Tree search: Θ(b^d); graph search with visited set: O(V+E). Frontier can be large.	Check whether generated/expanded nodes rise with reachable graph size and whether the queue becomes the largest memory structure.
Recursive DFS	Tree search worst case: Θ(b^m); graph search with visited set: O(V+E). Active stack follows one path.	Check whether DFS visits many states before finding the goal and whether its recursion depth becomes a limitation.
UCS	Tree bound depends on C*/ε; finite graph with binary heap: O((V+E) log V).	Check heap overhead, number of valid expansions and whether it returns the lowest path cost.

10.2 Interpretation

If the measured number of expanded nodes increases roughly with V + E on a fixed finite graph using duplicate detection, this supports the graph-search analysis. BFS may use more frontier memory because it stores many same-depth nodes. DFS can use less active-frontier memory, but the visited set still grows with explored states. UCS can take longer per expansion because heap insertion and removal require priority comparisons.

Measured values may differ from theoretical bounds because of graph connectivity, degree distribution, cycles, duplicate edges, goal position, neighbour ordering, equal-cost tie-breaking, Python data-structure overhead, garbage collection and operating-system scheduling. A single test does not prove a complexity bound; results across all four graph sizes should be compared using tables and, where useful, log-scale graphs.

10.3 Overall Comparison

For the hospital application, speed alone is not sufficient. A fast DFS route may have a high travel-time cost. BFS gives a minimum-edge route only when moves have equal cost. UCS is the most relevant comparison because the scenario explicitly assigns travel-time costs to moves. The final decision should therefore consider correctness, path cost, runtime and memory together.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

The recommended search strategy is Uniform-Cost Search (UCS) with an adjacency list, a min-priority queue and a best-known-cost table. The reason is that the hospital corridors and lifts have different travel-time costs. The robot should minimize total travel time rather than simply minimize the number of movements.

11.1 Reasons for Selecting UCS

- UCS expands the frontier state with the smallest accumulated travel-time cost.
- It returns an optimal route when all edge costs are non-negative.
- It handles different corridor and lift costs, unlike ordinary BFS.

- The cost and parent dictionaries prevent unnecessary repeated work and allow the route to be reconstructed.
- An adjacency list is suitable for a sparse hospital map because it stores only existing connections.

11.2 Comparison with Alternatives

Strategy	Strength	Limitation for this hospital
BFS	Simple; finds a route with the fewest edges.	May choose a slower route if edge travel times differ.
Recursive DFS	Can explore deep paths with a limited active path stack.	Does not guarantee a minimum-cost route; can explore an unhelpful branch and may exceed Python's recursion limit.
UCS (recommended)	Finds a minimum-total-cost route for non-negative travel-time edges.	Priority-queue operations and cost storage add overhead.

11.3 Practical Deployment

The hospital map should be built once as a weighted adjacency list and reused for multiple medicine deliveries. The robot should update the map when a corridor is blocked or a lift becomes unavailable. UCS should then plan using the latest edge costs. For a very large hospital, the implementation should use efficient heap operations, duplicate detection and route validation. The experimental results from Task 9 must be cited in the final submission to support the recommendation; the algorithm should not be selected only because of its Big-O notation.

Therefore, UCS is the best fit for the original fully localised, weighted-route problem. If all movement costs become equal, BFS may be a simpler alternative.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed requirement: After a power restart or sensor fault, the robot does not know its exact location. It knows only that it is at one of k possible junctions and must reach the pharmacy docking station regardless of its actual starting junction.

A. Impact Analysis – 2 Marks

The original problem assumed full observability and one known initial state. The changed problem is partially observable because the robot has uncertainty about its location. Its knowledge must now be included in the state. This set of possible locations is called a belief state.

For example, if the robot may be at junctions J1, J2 or J3, its belief state is B = {J1, J2, J3}. The robot cannot safely plan as if it were definitely at J1. It must choose actions and observations that account for every possible location.

B. Evaluation of the Original Design – 2 Marks

The original BFS, DFS and UCS algorithms can still be used after the robot's location is identified, but they cannot directly solve the uncertain-start problem by treating the belief set as one ordinary physical junction. The search state must become a set of possible locations rather than a single node.

If there are V physical locations, the number of possible belief states is at most 2^V, because every physical location may either be included or excluded from a belief set. If the robot knows it is in one of exactly k candidate junctions, the possible subsets of those candidates are at most 2^k. This exponential growth can make full belief-state search expensive.

C. Adaptation or Redesign – 2 Marks

The design should be modified to use local sensing followed by UCS. First, the robot stores the k possible locations in a belief set. It uses a camera, RFID reader, visual landmark or other local sensor to identify the actual junction. After receiving a reliable observation, it updates the belief set to the identified location. It then runs UCS from that known location to the pharmacy.

This approach is more practical than searching all belief states when local sensing is available. If sensing is unavailable or unreliable, a belief-state planner must choose a conditional plan that succeeds from every possible location. The general belief-state space can grow exponentially, up to 2^V.

D. Evidence-Based Defence – 2 Marks

A small experiment can compare k = 2, 4 and 8 candidate locations. The theoretical number of subsets is 2^k. With a perfect local sensor, the observation identifies one candidate and reduces the belief to a singleton. Without a local sensor, the planner must keep all possibilities and find a plan that is safe for every candidate. The table below shows the theoretical upper bound and the expected effect of sensing; actual generated-state counts and plan lengths depend on the hospital graph and sensor model.

k candidates	**Maximum subsets 2^k**	**After perfect local sensing**	**Without local sensing**
2	4	2 singleton beliefs; mean route length 1.5 edges; worst 2	4 belief states considered; worst-case route length 2 edges
4	16	4 singleton beliefs; mean route length 2.5 edges; worst 4	16 belief states considered; worst-case route length 4 edges
8	256	8 singleton beliefs; mean route length 4.5 edges; worst 8	256 belief states considered; worst-case route length 8 edges

12.1 Small Python Demonstration

The following experiment uses a simple corridor model: candidate junction J1 is one edge from the pharmacy, J2 is two edges away, and so on up to Jk. With a perfect local sensor, the robot identifies its actual junction and then follows the shortest route. Without a local sensor, the conservative route-length figure is the worst-case distance across all candidate junctions. The belief-subset count is 2^k.

```
for k in (2, 4, 8):
    belief_states = 2 ** k
    route_lengths = list(range(1, k + 1))
    mean_with_sensor = sum(route_lengths) / k
    worst_without_sensor = max(route_lengths)
    print(k, belief_states, k, mean_with_sensor, worst_without_sensor)

# Output:
# 2 4 2 1.5 2
# 4 16 4 2.5 4
# 8 256 8 4.5 8
```

The experiment produced belief-state counts of 4, 16 and 256 for k = 2, 4 and 8. With perfect sensing, the belief is reduced to one known location for each observation; the average shortest route lengths in the corridor model are 1.5, 2.5 and 4.5 edges. Without sensing, the conservative worst-case route lengths are 2, 4 and 8 edges. These are results from the explicitly defined simple corridor model, not measurements from a physical hospital robot. A different graph or sensor accuracy would produce different plan lengths.